

Applications of Maximum Flow

We have studied the maximum flow problem, and discussed various algorithms for computing a maximum flow in a network. We have already discussed that many different situations can be modelled as a flow network, such as fluid in pipes, currents in a circuit, or traffic in a computer network to name a few. As it turns out, there are also a surprising number of seemingly unrelated problems, that can be **reduced** into a maximum flow problem. Thus, efficient max-flow algorithms can be used to solve these problems. We look at some of such problems in this module.

Min Cut Problem

Given a flow network $G = (V, E)$, we wish to find a partition of V , (S, T) , having minimum capacity, i.e.

$$\min_{\text{cut } (S, T)} \sum_{\substack{u \in S, v \in T \\ (u, v) \in E}} c(u, v)$$

We have already discussed previously, that we can find a minimum cut by first finding a max-flow f of G , computing its residual graph G_f , and then finding the set $S = \{v \in V \mid v \text{ is reachable from } s \text{ in } G_f\}$ by doing a BFS from s . Then, $(S, V \setminus S)$ is a minimum cut.

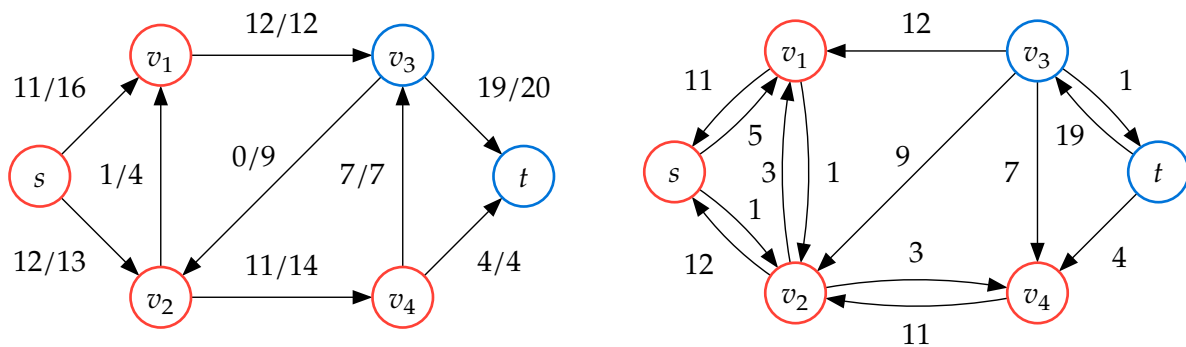


Figure 1: A flow network G with a max flow f . $S = \{s, v_1, v_2, v_4\}$. $(S, V \setminus S)$ is a min-cut of G .

Observe, that instead of giving an algorithm for the Min-Cut problem, we instead showed a procedure to solve a Min-Cut problem, provided that we have some way to solve an instance of the Max-Flow problem. In other words, we have shown that, if we have a subroutine to find a maximum flow in a flow network, we can use it to find a minimum cut in the flow network. We say, that **the minimum cut problem reduces in linear time to the maximum flow problem**.

Such a conversion of an instance of one problem X into an instance of another problem Y , such that the solution of the instance of problem Y can be used to get a solution of the instance of the problem X is called a **reduction**. In this case, since this conversion can be done in linear time, this is a **linear time reduction** (which is in fact a **polynomial time reduction**).

Maximum Bipartite Matching

Now we look at a completely different problem (seemingly unrelated to network flows), which is the bipartite matching problem. Before presenting the problem, let us define a few terms:

Given a graph $G = (V, E)$, a **matching** M in G is a subset of edges, such that no two edges share any common vertex.

A graph $G = (V, E)$ is said to be **bipartite**, if there exists a partition of V into A and B , $A \uplus B = V$ ($A \uplus B$ denotes a disjoint union, i.e. $A \cup B$ where A and B are disjoint sets.), such that all $e \in E$ are between $v_1 \in A$ and $v_2 \in B$. No edges connect vertices (v_1, v_2) such that $v_1, v_2 \in A$ or $v_1, v_2 \in B$.

The **maximum bipartite matching** problem: Given a bipartite graph $G = (V = A \uplus B, E)$ with $|A| = |B|$, find a maximum-cardinality matching in G .

A **perfect matching** in a graph G is a matching which covers every vertex of the graph. In a bipartite graph $G = (V = A \uplus B, E)$ where $|A| = |B| = n$, if a perfect matching exists, the cardinality of a perfect matching is n .

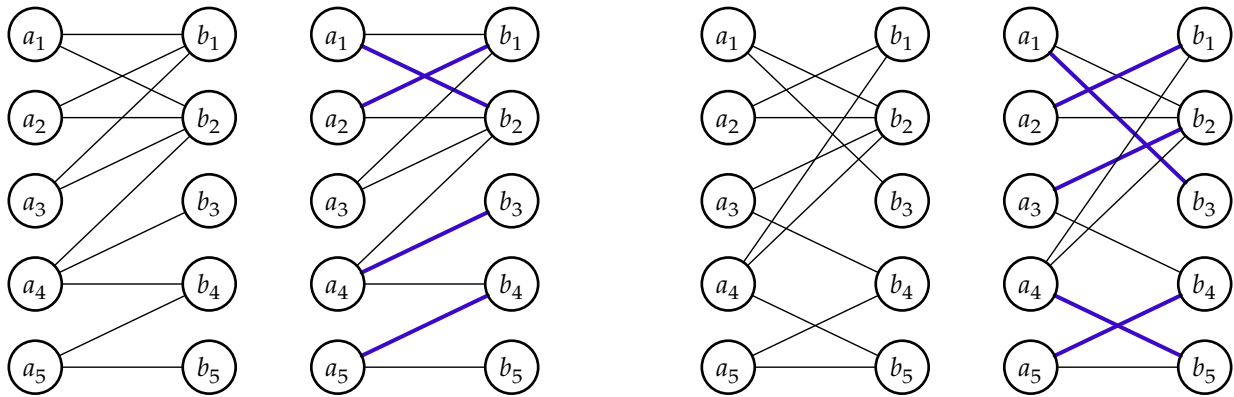


Figure 2: Bipartite graphs G_1 and G_2 , and maximal matchings M_1 and M_2 in them. $|M_1| = 4$, $|M_2| = 5$. G_1 does not have any perfect matching. G_2 has a perfect matching. (M_2 is one.)

Reduction to Max-Flow

We will show that the maximum-cardinality matching problem on bipartite graphs *reduces* in linear time to the maximum flow problem on flow networks.

To find a maximum-cardinality matching on a bipartite graph $G = (V = A \uplus B, E)$, we first define a directed graph $G' = (V', E')$, $V' = V \cup \{s, t\}$, $E' = \{(u, v) \mid (u, v) \in E, u \in A, v \in B\} \cup \{(s, a) \mid a \in A\} \cup \{(b, t) \mid b \in B\}$. Essentially, we add a source and a sink node, direct all edges from A to B , and add edges from the source to vertices in A , and from vertices in B to the sink. We give capacity 1 to all edges connected to s or t , and capacity ∞ to all other edges (ones that are between A and B).

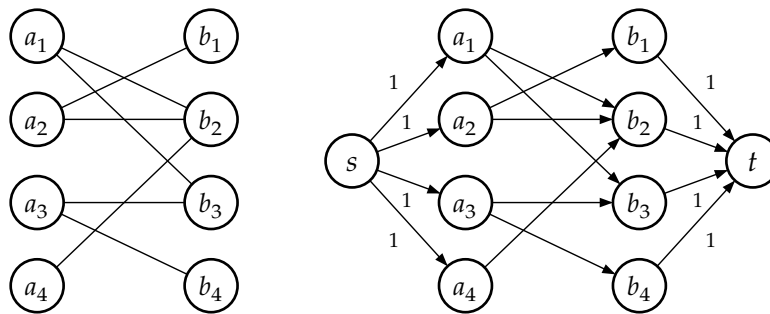


Figure 3: A bipartite graph G and its corresponding flow network G' as per our construction. Unlabelled edges have a capacity of ∞ .

We find a integral maximum flow f on G' .¹ Once we have this flow, choose $M = \{(u, v) \in E \mid f(u, v) = 1\}$, i.e. select the edges from G whose corresponding edges in G' have positive flow in f . Then, M is a maximum-cardinality matching of G .

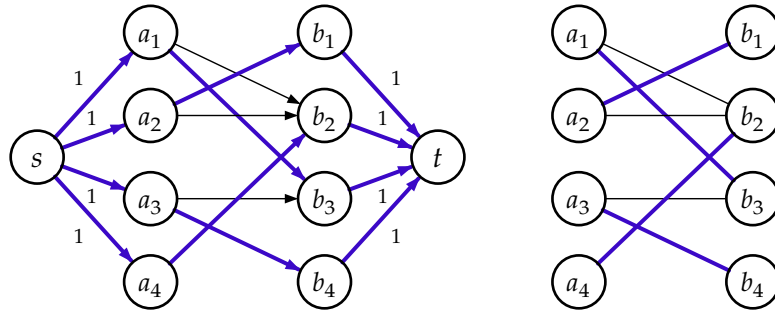


Figure 4: (a) Maximum flow of G' . Highlighted edges carry flow of 1. (b) A maximum cardinality matching M of G .

As always, we have to show that it works!

Maximum-cardinality bipartite matching reduces to Max-flow

Let $G = (A \uplus B, E)$ be a bipartite graph. Let $G' = (V', E')$ be the corresponding flow network constructed from G . The maximum cardinality of a matching in G equals the maximum value of an $s - t$ flow in G' .

Lemma 1: If G has a matching M of size k , then there exists a flow f in G' with $|f| = k$.

Proof: Let M be a matching of G , with $|M| = k$. Let $M = \{(u_1, v_1), \dots, (u_k, v_k)\}$. We can construct a flow f in G' :

- $f(s, u_i) = 1$ for all $i = 1, 2, \dots, k$
- $f(v_i, t) = 1$ for all $i = 1, 2, \dots, k$
- $f(u_i, v_i) = 1$ for all $i = 1, 2, \dots, k$
- For all other edges $(u, v) \in E', f(u, v) = 0$

Clearly, capacity constraints are satisfied. We need to show flow conservation:

- For some node $a \in A$,
 - if $a = u_i$ for some i , then its only incoming edge is (s, u_i) which has flow 1. One of its outgoing edges is (u_i, v_i) , which has flow 1. All other outgoing edges from a have 0 flow since they don't exist in M (by definition of a matching).
 - otherwise, its only incoming edge (s, a) has 0 flow. All its outgoing edges also have zero flow, since they don't exist in M .
- For some node $b \in B$,
 - if $b = v_i$ for some i , then its only outgoing edge is (v_i, t) which has flow 1. One of its incoming edges is (u_i, v_i) which has flow 1. All other incoming edges to b have 0 flow since they don't exist in M .
 - otherwise, its only outgoing edge (b, t) has 0 flow. All its incoming edges also have zero flow, since they don't exist in M .

¹All the max-flow algorithms we have discussed will necessarily find an integral max flow on a network with integral edge capacities. (In an integral flow, flow values on all edges are integral, not just the net value of the flow.)

Hence, f is a valid flow.

Value of the flow is the total outflow from s :

$$|f| = f(s, V - s) = \sum_{a \in A} f(s, a) = \sum_{i=1}^k f(s, u_i) + \sum_{\substack{a \in A \\ a \neq u_i}} f(s, a) = k + 0 = k$$

Lemma 2: If G' has an integral flow f with value $|f| = k$, then there exists a matching M in G with cardinality k .

Proof: Let G' have an integral flow f with value $|f| = k$. Define a matching M as follows:

$$M = \{(u, v) \in E \mid u \in A, v \in B, f(u, v) = 1\}$$

First let us show that flow in any edge in f will be either 0 or 1. Suppose there is some edge $(u, v), f(u, v) \geq 2$. Clearly, $u \in A, v \in B$. (any other type of edge does not have capacity to hold flow ≥ 2 .) Then, the outgoing flow from u is ≥ 2 . The only incoming edge to u is (s, u) which has a capacity of 1, which is a contradiction, since flow cannot be conserved in this case. So, $f(u, v) \in \{0, 1\}$ for all $(u, v) \in E'$.

To show M is a matching, we need to show that no vertex appears in more than one edge $e \in M$.

- Let $a \in A$. Suppose there are two edges (a, b_1) and (a, b_2) in M . This means, in $f, f(a, b_1) = 1, f(a, b_2) = 1$. So, total flow exiting a is ≥ 2 . The only incoming edge to a is (s, a) , which has a capacity of 1, which is a contradiction, since flow on a isn't conserved.
- Let $b \in B$ appear in more than one edge in M . Similarly, we can arrive at a contradiction.

Thus, $M \subseteq E$ is a valid matching.

Consider the cut $(s \cup A, B \cup t)$ in G' . Clearly, the flow through this cut is $|f| = k$. Then,

$$\begin{aligned} f(s \cup A, B \cup t) &= |f| = k \\ &\Rightarrow f(A, B) = k \\ &\Rightarrow \sum_{u \in A, v \in B} f(u, v) = k \end{aligned}$$

Since the flow in any edge can only be 0 or 1, the value of the last summation is the count of the edges $(u, v), u \in A, v \in B$, for which $f(u, v) = 1$. Therefore, M has k edges ($|M| = k$). ■

The Theorem follows from these two lemmas.

Perfect Matching

Given a bipartite graph $G = (A \uplus B, E)$, $|A| = |B| = n$, does G contain a perfect matching?

We can find this just by the above procedure, find the maximum cardinality matching in G , check if the cardinality of the matching is n , if so, then it is a perfect matching. If not, then no perfect matching exists. (Why?)

In other words, the decision problem “Does G have a perfect matching?”, is equivalent to checking if the value of the max flow of G' is n .

Observe that in Figure 3/4, G has a perfect matching.

Hall's Theorem

Here, we look at a characterisation of the above decision problem, i.e. Does G have a perfect matching?

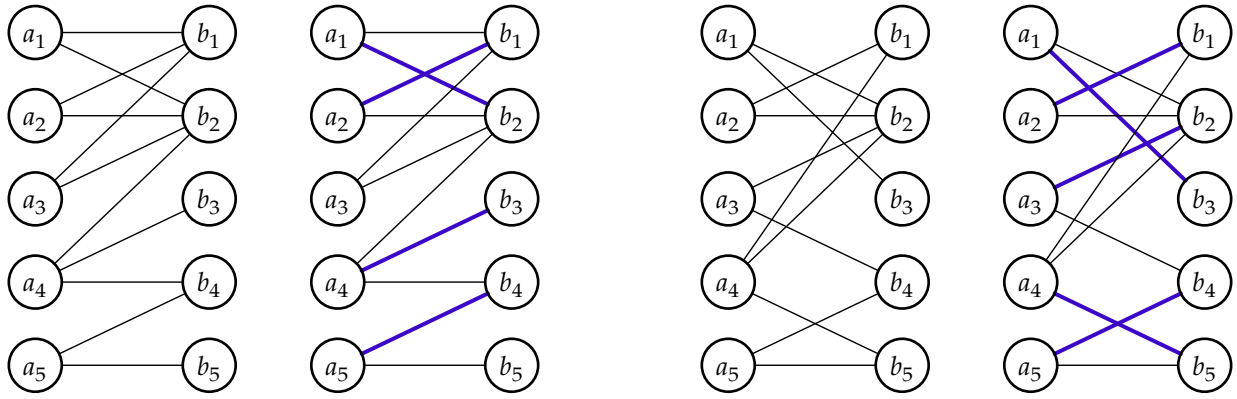


Figure 5: Graphs G_1 and G_2 , and their maximal matchings. (Same as Figure 2, reproduced here for the sake of your PDF scrolling sanity.)

Take a look at G_1 and G_2 in Figure 5. G_2 has a perfect matching, but G_1 does not. Could we give a succinct argument for why G_1 does not have any perfect matching, other than computing it via max-flow?

In the matching M_1 , a_3 is not matched with any vertex in B . If we did match a_3 with some vertex, say b_2 , then a_1 would become unmatched. The only other vertex that a_1 has an edge to is b_1 , say we match these. Now, a_2 becomes unmatched, and the only other vertex a_2 has an edge to is b_2 , but we have already matched it with a_3 . So it seems like we can never match all of $\{a_1, a_2, a_3\}$ in graph G_1 , however we try, one of these vertices gets left out.

Can we formalise this argument?

For some subset of vertices $S \subseteq A$, define the **neighbour set** of S : $N(S) = \{b \in B \mid \exists a \in S \text{ s.t. } (a, b) \in E\}$. So, $N(S)$ contains all vertices in B that has some edge with a vertex in S .

In G_1 , let $S = \{a_1, a_2, a_3\}$. Then, $N(S) = \{b_1, b_2\}$. Now, $|N(S)| < |S|$, which means the size of the neighbour set is smaller than the size of S itself. From this, it is very clear, that no perfect matching can exist in G . Because if it did, then the matched vertices of a_1, a_2, a_3 would be distinct, and they would be in $N(S)$, which clearly there aren't enough vertices in $N(S)$ for that to happen.

In general, in a graph G , if we find some $S \subseteq A$ such that $|N(S)| < |S|$, then G cannot have any perfect matching. In other words, for G to have a perfect matching, a necessary condition is: for all $S \subseteq A$, $|N(S)| \geq |S|$.

Hall's theorem states that this condition is a sufficient condition as well for existence of a perfect matching.

Hall's Theorem

A bipartite graph $G = (V = A \uplus B, E)$ with $|A| = |B| = n$ has a perfect matching if and only if $|S| \leq |N(S)|$ for every $S \subseteq A$.

Proof: $[\Rightarrow]$: Trivial.

$[\Leftarrow]$: Suppose the cardinality of a maximum matching in G is k . Construct the flow network G' from G by adding s and t , directing edges to go from A to B (all with capacity ∞), and adding

edges from s to $a \in A$, and from $b \in B$ to t (all with capacity 1). We know that the value of the max flow of G' is k .

By the max-flow min-cut theorem, there exists some cut (U, W) such that $c(U, W) = |f| = k$.

Let $A_1 = U \cap A, A_2 = A \setminus A_1; B_1 = U \cap B, B_2 = B \setminus B_1$. (Note that some of these sets may be empty.)

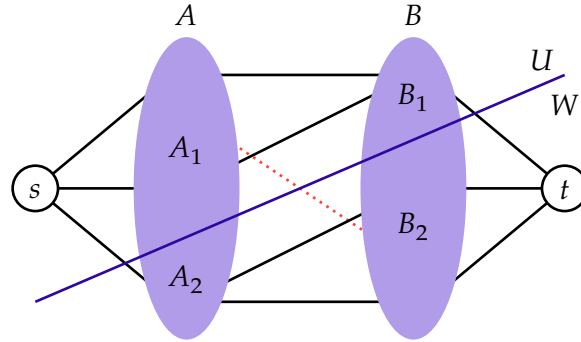


Figure 6: Defining A_1, A_2, B_1, B_2 by the min cut (U, W) . The red dotted edge cannot exist.

Since this is a cut with finite capacity, no edge having ∞ capacity must cross the cut. This means, for all $a \in A_1$, if (a, b) is an edge, then $b \notin B_2, b \in B_1$. So, $N(A_1) \subseteq B_1$. (Otherwise there will be an ∞ edge crossing the cut.)

Now, calculate the capacity of the cut (U, W) :

$$\begin{aligned}
 k = c(U, W) &= c(s \cup A_1 \cup B_1, A_2 \cup B_2 \cup t) \\
 &= c(s, A_2) + c(A_1, B_2) + c(B_1, t) \\
 &= |A_2| + 0 + |B_1| \\
 &= n - |A_1| + |B_1| \\
 &\geq n - |A_1| + |N(A_1)| && \text{since } N(A_1) \subseteq B_1 \\
 &\geq n - |A_1| + |A_1| && \text{since } |A_1| \leq |N(A_1)| \\
 &= n
 \end{aligned}$$

So, $k \geq n$, but we know that max-cardinality of a matching can't be $> n$, thus $k = n$. Hence, G has a perfect matching. ■

Kőnig's Theorem

This is another theorem that characterises the matching problem on bipartite graphs, into another problem, the **vertex cover problem**.

A **vertex cover** of a graph $G = (V, E)$ is a set of vertices $C \subseteq V$, such that for all edges $e \in E$, at least one end point of e must be in C . A **minimum vertex cover** of G is a minimum-cardinality vertex cover of G .

In a graph G , let M be a matching. Consider a vertex cover C of G . For each edge $e \in M$, at least one end point of that edge must be present in C (by definition of vertex cover). All endpoints of vertices in M are distinct. So, we have $|C| \geq |M|$, i.e. **the size of a maximum matching is a lower bound on the size of the minimum vertex cover of G** . Kőnig's theorem states that this lower bound is always achievable in bipartite graphs.

Kőnig's Theorem

The size of a minimum vertex cover is the same as the size of a maximum cardinality matching in every bipartite graph.

Proof: Let $G = (V = A \uplus B, E)$ be a bipartite graph. Let the cardinality of a maximum matching be k . Construct the flow network G' as usual, let (U, W) be its min cut, so $c(U, W) = k$.

Define $A_1 = U \cap A, A_2 = A \setminus A_1; B_1 = U \cap B, B_2 = B \setminus B_1$.

Again, since (U, W) is a cut with finite capacity, no edge in G must be from A_1 to B_2 . So, $N(A_1) \subseteq B_1$.

Here, if we calculate the capacity of the cut (U, W) , we have

$$\begin{aligned} k = c(U, W) &= c(s \cup A_1 \cup B_1, A_2 \cup B_2 \cup t) \\ &= c(s, A_2) + c(A_1, B_2) + c(B_1, t) \\ &= |A_2| + |B_1| \end{aligned}$$

So, $k = |A_2| + |B_1|$.

We claim, that $C = A_2 \cup B_1$ is a vertex cover of G .

Let us check all edges to see if one of their endpoints is in C .

For all $(u, v) \in E$,

- If $u \in A_1$, then $v \in N(A_1) \Rightarrow v \in B_1 \Rightarrow v \in C$.
- Else, $u \in A_2 \Rightarrow u \in C$.

So C is indeed a vertex cover of G . The size of this vertex cover is $|C| = |A_2| + |B_1| = k$, which is the size of the maximum cardinality matching of G . Since the size of a matching is a lower bound on the size of a vertex cover, C is a minimum vertex cover of G . ■

Summary

We have looked at some applications of the max-flow problem, how other problems can be reduced to the max flow problem, which gives us efficient algorithms to solve them. We have also seen some uses of flow networks to prove some theorems related to matching in bipartite graphs. Next up, we will look at the matching problem in general graphs.

Problems

where are they 🙄🙄